

Práctica 2:

Paralelización con MPI

En esta práctica se va a paralelizar el algoritmo evolutivo proporcionado en la Práctica 0 mediante MPI, estándar *de facto* para la programación paralela mediante paso de mensajes en sistemas con memoria distribuida. Para ello, se proponen una serie de cuestiones que se han de realizar de forma progresiva analizando en cada momento la mejora que se obtiene en el rendimiento. Esta práctica representa el 15% de la calificación de la asignatura.

La fecha de entrega de la práctica será el **7 de noviembre a las 23:55**. Entregas posteriores no se tendrán en cuenta. La entrega se realizará a través de la tarea habilitada en el Aula Virtual, adjuntando un fichero .zip o .tgz que contenga:

- **El código fuente paralelizado**, los ficheros *Makefile* y *run.sh* para su compilación y ejecución, así como otros ficheros de salida o con datos, que se podrán organizar **en diferentes carpetas** (una para cada cuestión). En el caso de que en alguna cuestión se requiera varias implementaciones de la misma rutina, se podrá entregar un único código fuente para esa cuestión con las distintas implementaciones en forma de comentarios. Además, el código ha de incluir los comentarios necesarios para ayudar a comprender las implementaciones realizadas. Debido a su elevado tamaño, **no se deberán incluir los ficheros de entrada generados en /input/**, solo **indicar en la memoria qué configuración de parámetros se ha usado en cada experimento**.
- **Un documento en formato .pdf**, con los nombres de los autores de la memoria de la práctica, así como el subgrupo al que pertenecen, donde se describa cómo se han resuelto las cuestiones propuestas, justificando las decisiones de paralelización tomadas en cada caso y analizando los resultados obtenidos. El código que se muestre debe coincidir con el que se entregue. **Se deberá incluir el sistema computacional** utilizado, que debe tener **como mínimo 6 núcleos físicos**, especificado con: `lstopo -f -p --no-legend --no-io sistema.svg`. Asimismo, el documento ha de incluir una sección en la que se indique cómo ha sido la coordinación, el reparto del trabajo entre los miembros del grupo y el tiempo dedicado.

En aquellos casos en los que sea necesario mostrar la evolución de las prestaciones, se puede estructurar la información en tablas, incluyendo una leyenda que indique la instancia del problema sobre la que se realizan los experimentos, así como el resto de parámetros de entrada:

n_procesos | *t_ejec* | *speed-up* | *eficiencia*

Tabla 1. Leyenda para la tabla considerando *n*, *m*, *n_gen*, *tam_pob*.

donde, en este caso, para calcular la eficiencia, consideraremos los procesos MPI puestos en marcha como elementos de proceso.

Se recomienda la creación de gráficos (con su leyenda correspondiente) a partir de las series de valores más relevantes de las tablas, en los que se muestre la evolución de las métricas al variar el número de procesos.

En general, para obtener unos resultados más fiables es aconsejable, en cada experimento, realizar la media de varias ejecuciones. Además, se deberá indicar el algoritmo utilizado para tomar el tiempo secuencial a la hora de calcular el speed-up, pudiendo ser, por ejemplo, el algoritmo secuencial o el algoritmo paralelo con 1 proceso.

Algoritmo con paso de mensajes. El algoritmo paralelo se implementará siguiendo un modelo de islas (Figura 1). Cada proceso (incluido el maestro) trabajará con su propia isla (subconjunto de la población inicial) y llevará a cabo de manera independiente la evolución de su subpoblación. Se hará uso de una variable, NGM, que determinará cada cuántas generaciones se ha de producir la migración de NEM individuos de cada isla, la mezcla en el proceso 0, su ordenación y el posterior envío de parte de la población (los mejores, por ejemplo) a cada isla para continuar el proceso evolutivo.

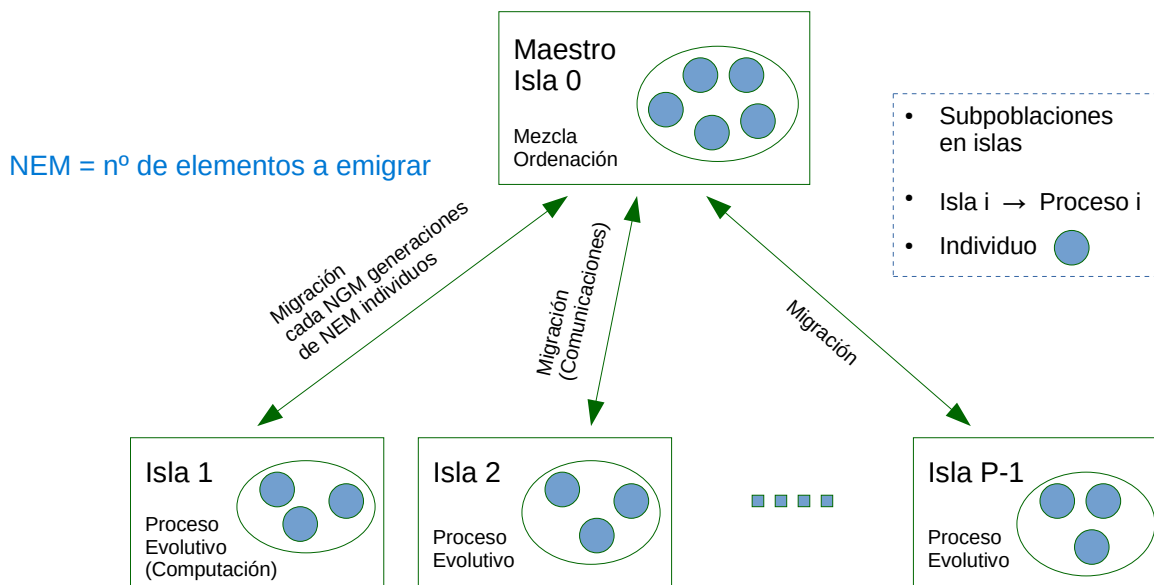


Figura 1: Modelo de islas.

La estructura paralela del algoritmo de paso de mensajes es sencilla: cada proceso ejecuta en paralelo una copia del algoritmo secuencial (el de la P0) aplicado a su subpoblación (modelo SPMD: desde que se lanza *mpirun*, cada proceso ejecuta su copia del programa entero y mediante estructuras condicionales que dependan de su identificador, cada proceso puede realizar computación diferente dentro del programa). Para completar este modelo simple, se establecen comunicaciones entre los procesos para compartir información (migración de subpoblaciones).

A continuación, se muestran las cuestiones a realizar indicando de forma orientativa su distribución en cada semana.

Hay que tener en cuenta que se proporciona, anexo, un fichero *derivados_mpi.c* con la función que permite crear el tipo de datos derivado MPI para el envío y recepción de datos de tipo *Individuo*, que será el usado en las comunicaciones implementadas en las diferentes cuestiones, salvo que se indique otra cosa. Para que MPI pueda utilizar el tipo *Individuo*, es necesario modificar el campo *array_int* del tipo de datos definido en el fichero *mh.h* para que sea un array de longitud estática (con tamaño máximo suficiente para los problemas concretos sobre los que se aplique el algoritmo).

Hay que enviar la estructura de un individuo entera, fitness y array

b) que la población sea un array de individuos en vez de punteros a individuos para que estén contiguos en memoria: Individuo en vez de *Individuo

1ª Semana

Cuestión 1.

- a) Indicar qué pasos habría que llevar a cabo para paralelizar el algoritmo con MPI (no es necesario indicar las primitivas de comunicación). Puedes desarrollar la cuestión de dos maneras: (1) escribir las modificaciones que deberían realizarse sobre cada parte del código secuencial para implementar el modelo de islas, o (2) elaborar un esquema en pseudocódigo que resuma el paralelismo implementado en este modelo. En cualquier caso, tener en cuenta el patrón de computación y comunicaciones entre procesos, desde la lectura inicial de parámetros de entrada, el intercambio de información entre procesos durante el desarrollo del algoritmo, la computación en paralelo, y la comunicación final de resultados.

- ✓ b) Las primitivas como *MPI_Send* para comunicaciones con paso de mensajes pueden enviar arrays de datos del mismo tipo a condición de que estos estén contiguos en memoria. El código actual implementa la población como un array de punteros a *Individuo* con lo que los datos pueden estar en posiciones no contiguas de memoria. Como consecuencia se puede producir un error en el envío de poblaciones o subpoblaciones de individuos. Modifica el código como consideres mejor para solucionar este inconveniente, teniendo en cuenta que puedes cambiar de población de **Individuo* a *Individuo* o reservar memoria dinámica auxiliar contigua y apuntar a ella para crear la población.

- ✓ **Cuestión 2.** El maestro atiende a los esclavos en el orden en que llegan **BLOQUEANTE**. Paralelizar el algoritmo evolutivo utilizando primitivas de **comunicación síncrona**. Hacer uso de las constantes *MPI_ANY_SOURCE* o *MPI_STATUS_IGNORE*, si se considera conveniente, para evitar que el proceso 0 quede ocioso durante la recepción de mensajes del resto de procesos. Analizar las prestaciones que se obtienen al variar el número de procesos. Tener en cuenta el punto (3) en Observaciones para determinar los valores de NEM y NGM.

2ª Semana Cada NGM dentro del bucle principal de AG realizar la migración del esclavo al maestro con NEM, nº de elementos a migrar. Comparar t_{ejc} y fitness para ver con qué valores de NGM Y NEM se comporta mejor. $I = fit/t_{ejc}$. Cuando mayor sea I mayor bondad del algoritmo, mejor.

- ✓ **Cuestión 3.** Modificar la cuestión anterior para que el envío y la recepción de datos se realice mediante el uso de las funciones *MPI_Pack* y *MPI_Unpack* para **empaquetamiento de datos** (o cualquier otro método de envío diferente al uso del tipo derivado *Individuo*, de forma que se envíe en un mismo mensaje toda la estructura de individuo, no varios mensajes con cada parte). Presentar también resultados de los experimentos realizados.

- ✓ **Cuestión 4.** **NO BLOQUEANTE**. Paralelizar el algoritmo evolutivo utilizando primitivas de **comunicación asíncrona**. Analizar las prestaciones obtenidas al variar el número de procesos.

Distribución inicial: Scatterv

3ª Semana

Migración: buffer_mezcla con Gather--> Distribución de nem con Bcast

Recolección final con Gather

Mirar observación 5

- ✓ **Cuestión 5.** Paralelizar el algoritmo evolutivo utilizando primitivas de **comunicación colectiva**. Analizar las prestaciones obtenidas al variar el número de procesos.

Cuestión 6. Elegir la mejor configuración de parámetros y tipo de comunicaciones para paralelizar la totalidad del algoritmo. Justificar las decisiones tomadas. De este modo, se pretende tener un código paralelo óptimo en memoria distribuida que se podrá usar para comparar con el código secuencial, y paralelo en memoria compartida e híbrido.

Cuestión 7. Realizar el informe final incluyendo las conclusiones generales y valoración personal sobre el trabajo realizado.

Observaciones

- 1) El programa principal (*main.c*) tiene que modificarse para que permita inicializar y finalizar el entorno de ejecución de MPI, así como llevar a cabo la toma de tiempos de forma adecuada, por ejemplo, si es necesario, mediante barreras de sincronización (*MPI_Barrier*).
- 2) El procedimiento para implementar el algoritmo de paso de mensajes queda a elección del alumno. No obstante, puede ser recomendable realizar pruebas básicas iniciales con MPI. Por ejemplo, se puede utilizar el código *hello.c*, de la Sesión 4 de Teoría, para **enviar un array de enteros** con comunicaciones punto a punto. Después, **copiar las funciones *crear_tipo_datos* y *crear_individuo* para enviar individuos en lugar de enteros**. También probar con *broadcast*, *scatter*, *gather*...

Posteriormente, comenzar la implementación del algoritmo con un código simple, por ejemplo, con solo dos procesos y una iteración, donde **solo se envíe un *Individuo*** con comunicaciones punto a punto. Después, aumentar progresivamente la complejidad del código, **enviando un array de individuos**, aumentar el número de procesos, con varias iteraciones donde solo haya migraciones al final desde cada isla al maestro. Por último, aumentar la frecuencia de las migraciones considerando los parámetros NGM y NEM, hasta implementar el algoritmo completo necesario para resolver las cuestiones de la práctica.

- 3) Una vez implementado el esquema de comunicaciones de la Cuestión 2, se pueden **lanzar ejecuciones variando NGM y NEM** para determinar qué valores permiten obtener buenos resultados (de fitness, tiempo o ambos), con el objetivo de fijar estos parámetros adecuadamente. En cualquier caso, la mejor configuración dependerá de la instancia de problema, de la implementación del algoritmo realizada (incluyendo el tipo de comunicaciones) y del sistema computacional.

Además, se pueden hacer pruebas variando NGM y NEM para conseguir unos tiempos de computación y de comunicaciones similares (del mismo orden de magnitud) en cada iteración, pudiendo variar también, para este fin, el tamaño de la población o del individuo. Por ejemplo, si tenemos que, inicialmente, la computación es mucho mayor que las comunicaciones, podemos **disminuir NGM** lo que **aumentaría la frecuencia de las migraciones (más comunicaciones)** y si, además, disminuimos la computación en cada isla (subpoblaciones más reducidas o valor del parámetro *m* menor), haría que los tiempos tiendan a igualarse. Así, se puede ver la influencia de estos parámetros en los tiempos de ejecución del algoritmo. Una vez realizadas estas pruebas, se pueden fijar unos valores de NGM y NEM adecuados para contestar todas las cuestiones de la práctica.

Por otro lado, también se podría considerar que **NEM** no es una constante a determinar inicialmente, sino que puede variar en cada ejecución y podría tener valores como: el tamaño de toda la subpoblación (puede ser lo mejor para que haya la máxima contribución de comunicaciones al tiempo total, pero no necesariamente lo mejor para el fitness), o el de un porcentaje de esta. Por supuesto, como se ha indicado previamente, también se le podría dar un **valor constante**, razonablemente adecuado, promedio, útil para todos los experimentos.

Como esta parte queda abierta al criterio particular, el alumno deberá elegir la implementación que considere más adecuada, razonando y analizando los resultados obtenidos.

- 4) Los experimentos no tienen por qué limitarse a usar un número de procesos máximo igual al número de cores físicos. De hecho, si se supera este máximo puede llegar a observarse como se degrada el rendimiento del algoritmo. En caso de que en una máquina concreta no esté habilitado por defecto, se puede usar la opción `-oversubscribe` para forzar a MPI a usar más procesos que cores físicos disponibles. Si el número de procesos es inferior o igual al número cores físicos, esta opción no tiene efecto.
- 5) En comunicaciones colectivas, cuando el tamaño de la población no es divisible entre el número de procesos, existe la opción de usar `MPI_Gatherv` o `MPI_Scatterv`, variantes de sus respectivas primitivas habituales, que permiten a los mensajes recibidos o enviados tener diferentes longitudes. También se puede usar `MPI_Gather` o `MPI_Scatter` con la opción `MPI_IN_PLACE` para el proceso 0, y para el resto, una llamada estándar a estas primitivas.
- 6) Para comparar mejor las prestaciones del algoritmo entre implementaciones, MC, PM e Híbrida, puede ser interesante comparar no solo el tiempo, sino un indicador I (por ejemplo, $I = \text{fitness} / \text{tiempo}$) que englobe tiempo y fitness. De esta forma, habría que tantear mejor parámetros como NEM o NGM para mejorar también el fitness. Es evidente que, si solo se desea optimizar el tiempo, lo mejor sería $NEM=1$ y $NGM = n_{gen}$ (la menor cantidad de individuos y la menor frecuencia de migración). Otra opción más simple, si no se quiere usar un indicador, podría ser medir el fitness alcanzado fijando un tiempo de ejecución razonable, igual para todas las implementaciones.